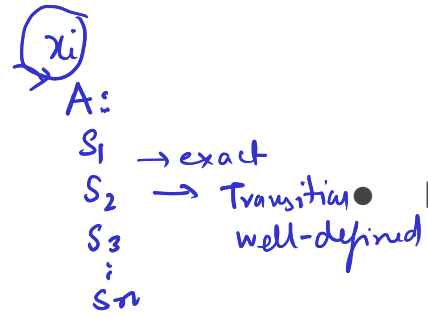
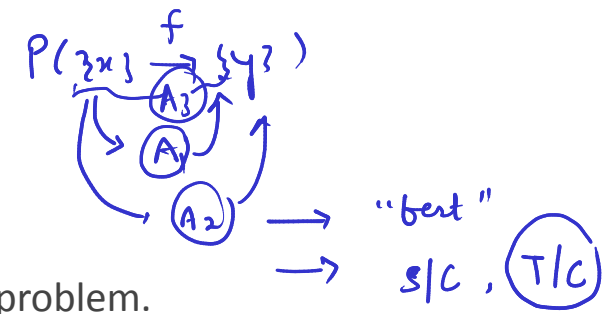


Algorithms

Algorithms

- An exact formal description of the steps to solve a computational problem.
 - An exact description of the steps to calculate the corresponding function.
- An algorithm is a sequences of unambiguous instructions for solving a problem. Algorithms must be **finite, complete, and correct**.

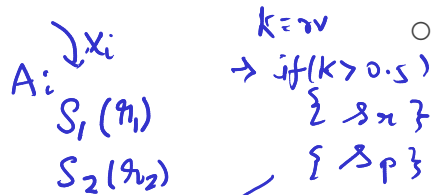
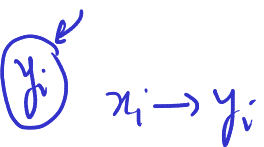


Deterministic algorithms

- Computation steps and the result of algorithm
 - are same for each run on a particular input

Randomized algorithms

- Computation steps are **mapped to random variable(s)**
- Order of execution or the result of the algorithm
 - might be different for each run on the same input



random-time
correct answer

constant-run-time
approximate answer

$$\rightarrow P(\{x\}, \{y\})$$

$$\underline{y_i} = \underline{f}(\underline{x_i})$$

f : functional
 A : state based sequence
 $P \equiv \{variables\}$

$$x_i \rightarrow \boxed{A} \rightarrow y_i$$

$$\underline{A}: \underline{S_1}, \underline{S_2}, \underline{S_3} \dots \underline{S_n} \rightarrow \underline{qP}$$

exact, formal description
finite

1. Probabilistic Analysis and Randomized Quicksort, Lecture Notes, Manuel Blum, CMU 15-451, Spring 2007
2. Randomized algorithm for approximate median, Advanced Algorithms, Mark de Berg, 2017

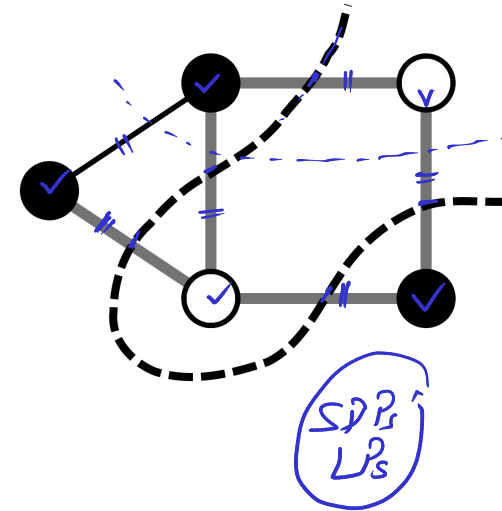
Algorithms for Optimization Problems

• Maximum-Cut ✓

$$G = (V, E), w : E \mapsto \mathbb{R}^+ \text{ and } S \cup \bar{S} = V$$

$$\text{obj fn} \rightarrow \sum_{u \in S, v \in \bar{S}, (u,v) \in E} w_{uv}$$

exact
 $O(2^n)$



$e_1 e_2 \dots e_n$
 (2^n)
 $\Rightarrow$ "exact"
 $O(L^n) \rightarrow$

✓ Approximate algorithms $\rightarrow$ Polynomial.

3-factor approx. algo.
 $(1/3)$

Compute an approximate in at-most polynomial time

$\rightarrow$ does not guarantee the optimal solution

$\rightarrow$ provides provable guarantee for finite-factor approximation

• Heuristics methods

○ No provable guarantee for optimality but useful for certain problems

$\rightarrow$ **Constructive algorithms:** Greedy methods $\rightarrow$ next-best solution

$\rightarrow$ **Local search algorithms:** Hill-climbing methods

• Metaheuristic methods $\rightarrow$ "almost near-optimal"

○ Strategy-guided heuristics for solving optimization problems

$\rightarrow$ **Trajectory methods:** Simulated Annealing, Tabu Search

$\rightarrow$ **Population-based:** Evolutionary computation $\rightarrow$ SA

Exercise:

1. Learn about Max-Cut
2. Learn about 2-approximate TSP

MSTs

